

# Relatório de Execução - Vector Addition (OpenMP)

Este relatório detalha os resultados da execução do programa de adição de vetores (Vector Addition) na arquitetura CPU versus os métodos de offloading para GPU (baseados nos Slides 27 e 48).

## 1. Ambiente e Submissão

O job foi submetido ao cluster NPAD, utilizando a fila/partição `gpu-8-v100`, utilizando a placa de vídeo V100 disponível. O código foi compilado utilizando o `nvc` (NVIDIA HPC SDK versão 24.11) com a flag `-mp=gpu` para habilitar o offloading de OpenMP para a GPU.

## 2. Comparação de Desempenho

| Versão                             | Tempo de Compute (s) | Tempo Total (s) | Erros |
|------------------------------------|----------------------|-----------------|-------|
| <b>CPU (Baseline)</b>              | 0.011s               | 0.071s          | 0     |
| <b>Slide 27 (Target Básico)</b>    | 0.301s               | 0.361s          | 0     |
| <b>Slide 48 (Target Otimizado)</b> | 0.318s               | 0.385s          | 0     |

### Análise dos Tempos

- **CPU (Baseline)** apresentou o melhor desempenho.
- **GPU (Slide 27 e 48)** demorou em torno de 0.30 a 0.31 segundos na etapa de “Compute”, sendo significativamente **mais lenta que a CPU** para este caso.

**Por que a GPU foi mais lenta?** A operação de adição de vetores ( $c[i] = a[i] + b[i]$ ) possui baixíssima “intensidade aritmética” (poucas operações matemáticas por byte de memória lido). Ao utilizar o offloading, ocorre o gargalo da transferência de dados entre a memória do Host (CPU) e do Device (GPU) pelo barramento PCIe. Além disso, há o tempo adicional (overhead) necessário para inicializar a GPU e lançar o kernel. Para um  $N$  (tamanho do vetor) em que a computação em si é quase instantânea na CPU, o tempo das transferências domina completamente o tempo de execução na GPU.

### Comparação Slide 27 vs Slide 48

- **Slide 27:** Utiliza diretivas implícitas (apenas `#pragma omp target` e `#pragma omp loop`). O compilador automaticamente deduziu as movimentações de dados e a distribuição de laços para a GPU.
- **Slide 48:** Utiliza mapeamento e particionamento de threads explícitos (`#pragma omp target teams distribute parallel for map(...)`). Como se nota, ambos tiveram o tempo muito parecido (com variações mínimas provavelmente oriundas da própria latência do hardware na medição), visto que o compilador `nvc` faz um trabalho muito bom em paralelizar adequadamente até mesmo a versão mais genérica do Slide 27.

### 3. Problemas e Tolerância

- **Erros de Tolerância:** Não foram reportados erros (“0 errors” em todas as execuções). O nível de tolerância do código ( $TOL = 0.0000001$ ) foi validado satisfatoriamente na precisão de float.
- **Problema de Stack Size Potencial:** Note que o código em .c original aloca 4 vetores de 10.000.000 de floats diretamente na “stack” (`float a[N], b[N], c[N], res[N];`), o que soma aproximadamente 160 MB de memória. Isso costumeiramente gera `Segmentation Fault` (estouro de pilha) em sistemas comuns, sendo bem-sucedido aqui puramente devido à configuração do SLURM/NPAD que fornece *stack size ilimitado* nas partições. O ideal seria sempre utilizar alocação dinâmica (com `malloc`) na “heap” para tamanhos grandes.
- **Módulo e Compilador:** Foi identificado um erro com a ferramenta `module` do NPAD que impedia a utilização de `module load nvhpc`. Como solução, o path para o compilador `nvc` foi injetado diretamente no script `job.sh` apontando para os binários em `/opt/npad/shared/compilers/nvidia_hpc_sdk/24.11`.